CampusHub — Web Application Development Project Report

1. Abstract

CampusHub is a responsive student task planner that centralises coursework, due dates, priorities, and study progress. This edition upgrades the initial HTML/CSS/vanilla-JavaScript and native HTTP-server prototype to a React frontend, Express REST API, and MongoDB persistence. The design uses a flat green, red, white, and black palette with square corners and responsive desktop/mobile layouts.

2. Objectives

- Organise tasks by course, due date, and priority.
- Provide create, read, update, delete, complete, and reopen actions.
- Restore the calendar, course overview, analytics, search, filtering, sorting, and settings as functional views.
- Demonstrate all five Web Application Development course outcomes.
- Prepare a reproducible local setup and Render deployment path.

3. Technology Stack

- Frontend: semantic HTML rendered by React, CSS, JavaScript (ES modules), Vite.
- React concepts: reusable components, controlled inputs, event handlers, `useState`, `useEffect`, `useMemo`, and `useCallback`.
- Backend: Node.js and Express, JSON REST endpoints, validation middleware/handlers, CORS, error responses.
- Database: MongoDB through Mongoose; indexed task schema.
- Delivery: npm scripts, `.env.example`, Render Blueprint (`render.yaml`), and Postman collection.

4. Architecture

The browser renders the React application. Frontend API functions send JSON requests to `/api/tasks`; Express validates the request and performs operations through the Mongoose Task model. MongoDB stores the persistent task records. The former JSON file is retained only as a migration source and is not used as the live database.

5. Functional Modules

1. **Dashboard:** total, active, completed and overdue metrics; upcoming deadlines; completion progress; high-priority focus list.

2. **Task management:** create/edit modal, completion and reopening, deletion, title/course/notes search, course and priority filters, and sorting.

3. **Calendar:** month navigation, today shortcut, task markers, selected-date agenda.

4. **Courses:** group tasks by subject with course-level completion progress.

5. **Analytics:** completion rate, high-priority count, upcoming week count, priority distribution, and subject workload.

6. **Settings:** light/dark theme and delete-completed operation.

7. **API and persistence:** MongoDB CRUD, validation, health check, JSON error responses, and legacy JSON migration.

6. API Summary

- `GET /api/health`
- `GET /api/tasks` (optional `status=active|completed`)

- `GET /api/tasks/:id`
- `POST /api/tasks`
- `PATCH /api/tasks/:id`
- `DELETE /api/tasks/:id`

Invalid payloads return HTTP 400, unknown task IDs return 404, created tasks return 201, and database unavailability returns 503 for task routes. See `postman/CampusHub.postman_collection.json` for runnable request examples.

7. Course Outcome Mapping

- **CO1:** responsive layouts, semantic form labels, accessible focus indicators, HTML/CSS/JavaScript interactions.
- **CO2:** React components, hooks, controlled forms, event-driven updates, and data-loading effects.
- **CO3:** Express REST API with CRUD methods, validation, status codes, and Postman examples.
- **CO4:** Mongoose schema and MongoDB persistence with environment-based credentials and a safe, rerunnable JSON migration script.
- **CO5:** production build/start scripts and Render configuration with MongoDB Atlas setup instructions.

8. Installation and Verification

1. Install Node.js 20+ and npm.
2. Run `npm install`.
3. Copy `.env.example` to `.env`; set `MONGODB_URI`.
4. Run `npm run dev`; open `http://localhost:5173`.
5. Run `npm test` for repository contract checks.
6. Import the Postman collection and execute its requests against the running API.
7. For production, run `npm run build`, set `NODE_ENV=production`, and start with `npm start`.

The automated repository checks verify expected frontend modules, API route definitions, model presence, and migration safeguards. They do not replace browser testing, database-backed integration tests, a live Postman run, or live Render deployment verification. Those steps require dependencies to install and a reachable configured MongoDB instance.

9. Migration and Deployment Safety

`npm run migrate:json` imports the existing `data/tasks.json` file without deleting it. The migration tracks legacy IDs to avoid duplicate imports on reruns. For Render, configure `MONGODB_URI` as a secret, configure Atlas network access carefully, and verify health plus create/update/delete/persistence on the deployed service before claiming successful deployment.

10. Conclusion

CampusHub now has a full-stack architecture appropriate for demonstrating CO1–CO5 while preserving the calendar, courses, analytics, task CRUD, search, filtering, sorting, and settings. Deployment and database-backed verification remain environment-dependent and must be completed with actual credentials.